Algorytmy sortowania

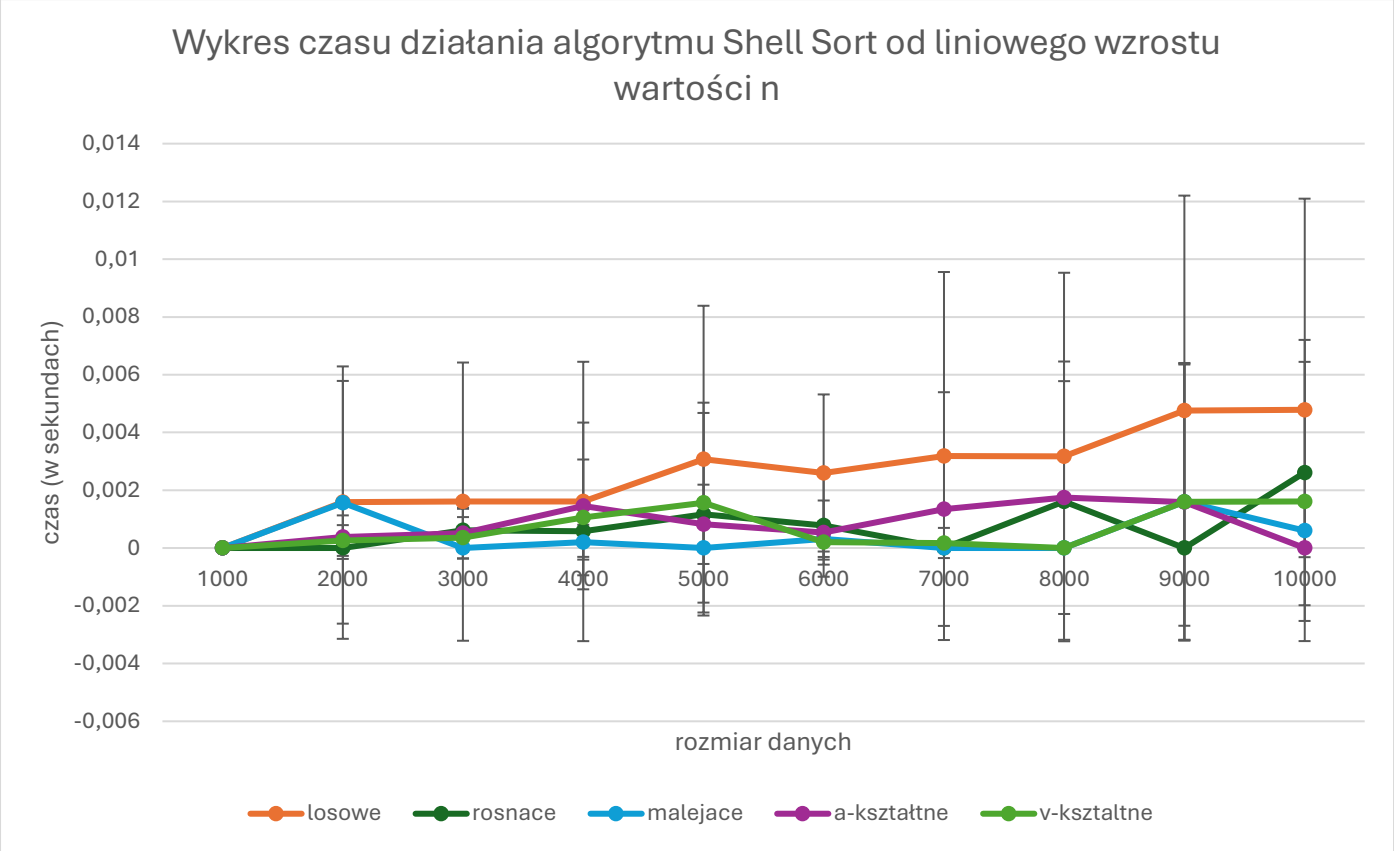
Maria Linkowska 167906, Sara Rachmańczuk 167898

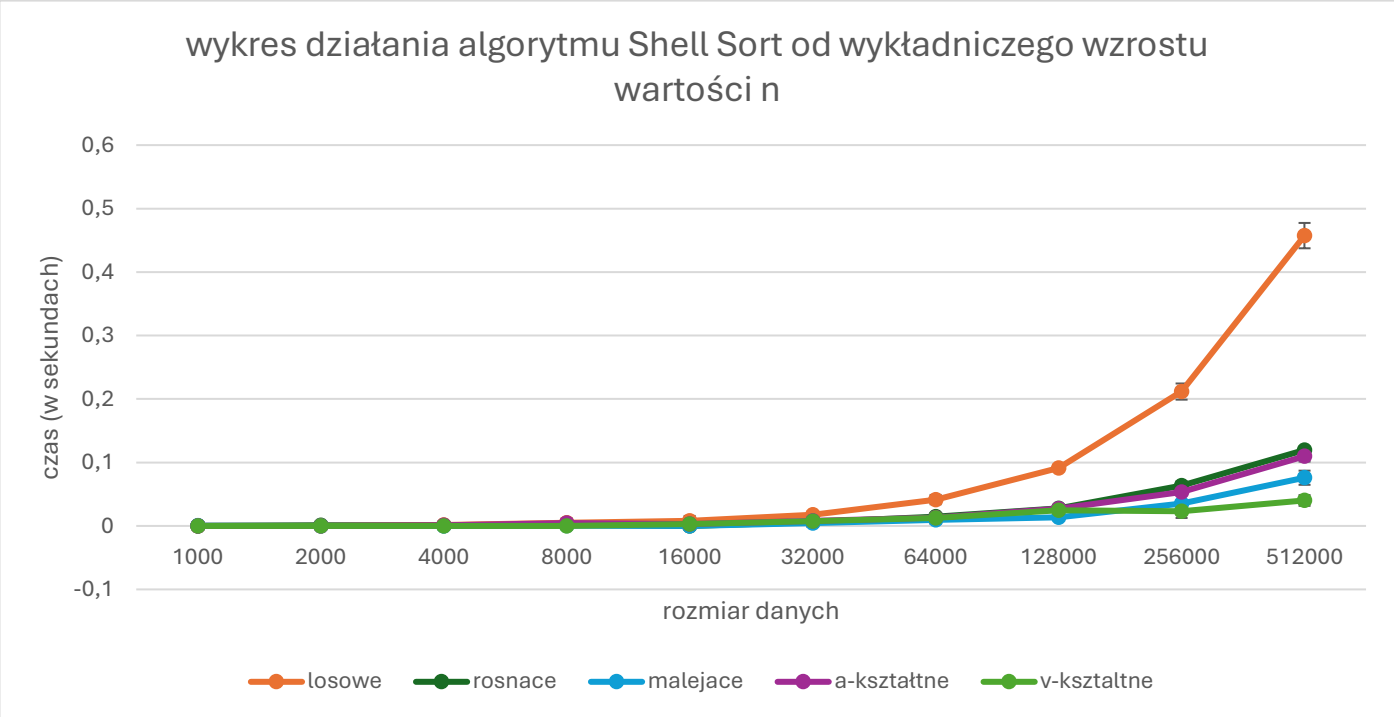
1. **Celem** tego zadania była analiza porównawcza 5 algorytmów sortowania: Shell Sort, Merge Sort, Heap Sort, Quick Sort rekurencyjny i iteracyjny. Badanie to miało na celu sprawdzenie jak każdy z tych algorytmów radzi sobie z różnymi typami danych wejściowych oraz jak zmienia się ich czas wykonania wraz ze wzrostem liczby elementów.

2. **Algorytmy**:
    a. **Shell Sort** – to ulepszona wersja sortowania przez wstawianie (Insertion Sort), która zwiększa efektywność poprzez porównywanie elementów oddalonych od siebie o określony odstęp. Zamiast sąsiadów, algorytm sortuje mniejsze podgrupy elementów, stopniowo zmniejszając odstęp do 1, co ostatecznie przypomina klasyczne wstawianie, ale na już częściowo uporządkowanych danych. Jego złożoność czasowa wynosi O(n log n).
    b. **Merge Sort** – czyli sortowanie przez scalanie to efektywny algorytm sortowania oparty na metodzie „dziel i zwyciężaj". Działa poprzez rekurencyjny podział tablicy na mniejsze podtablice aż do uzyskania jednoelementowych fragmentów, które następnie są scalane w posortowaną całość. Jego złożoność wynosi O(n log n).
    c. **Heap Sort** – tzw. sortowanie przez kopcowanie. Algorytm używa kopca binarnego i jest to algorytm sortowania w miejscu. Jego główną zaletą jest stabilna złożoność czasowa, która zawsze wynosi O(n logn) oraz nie pochłania zbyt dużo pamięci.
    d. **Quick Sort rekurencyjny** – to rekurencyjny algorytm sortowania oparty na metodzie „dziel i zwyciężaj". Działa poprzez wybór elementu osiowego (pivot) i partycjonowanie tablicy na mniejsze i większe elementy, które następnie są sortowane rekurencyjnie. Zapewnia średnią złożoność czasową O (n log n), natomiast jego pesymistyczna złożoność może wynosić nawet $O(n^2)$.
    e. **Quick Sort iteracyjny** – opiera się na metodzie „dziel i zwyciężaj", wykorzystuje jawny stos do przechowywania zakresów tablic - dzięki temu unika ryzyka przepełnienia stosu wywołań dla dużych danych, zachowując średnią złożoność czasową (O(n logn)).

3. **Testy** wykonane zostały dla wielkości n od 1000 do 10000 (skok liniowy) oraz dla wartości wykładniczych. Wyniki pobrano z 10 prób by zminimalizować błąd pomiarowy. Badania te zostały przeprowadzone dla kilku rodzajów danych:
    a. Losowe
    b. Rosnące
    c. Malejące
    d. A-kształtne
    e. V-kształtne

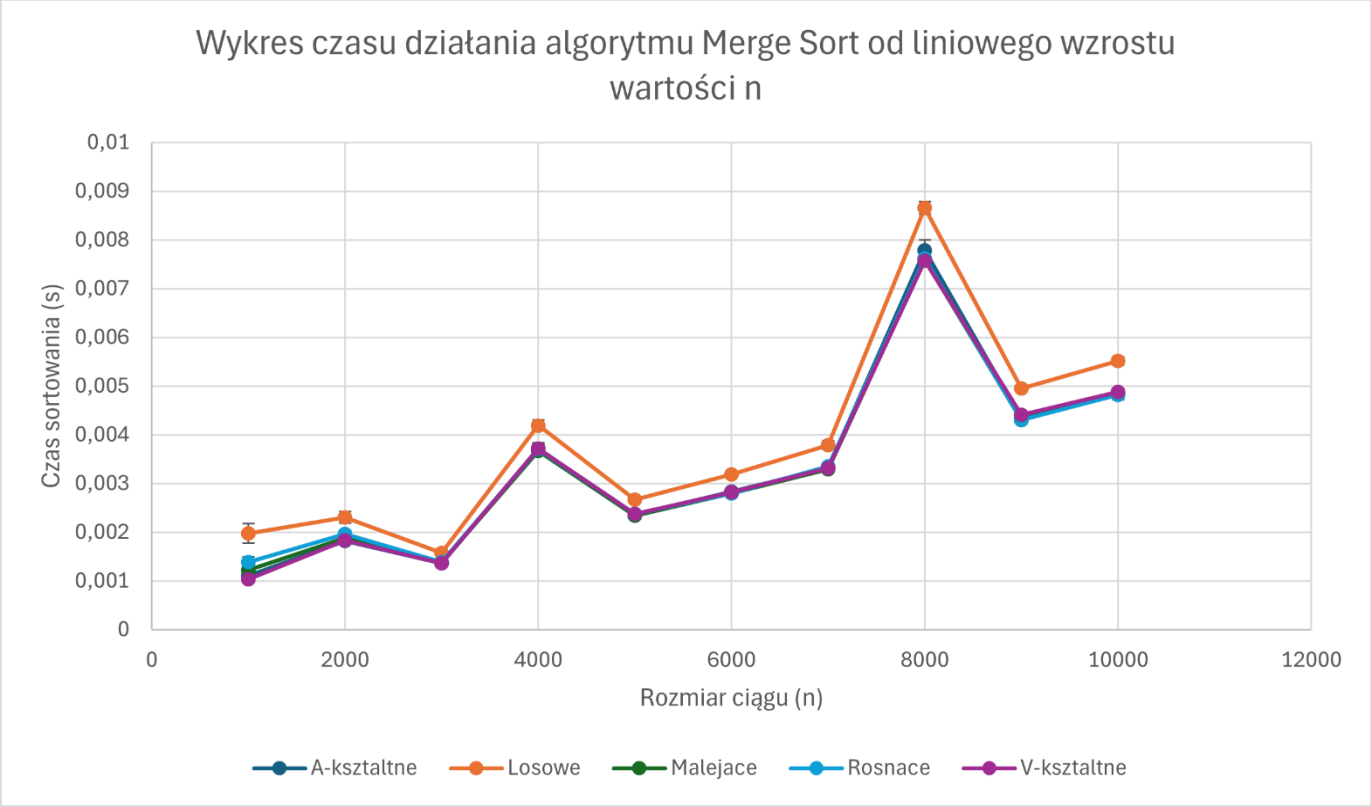
4. **Analiza wyników** – dla każdego algorytmu sprawdzenie jak się sprawdzają dla konkretnych typów danych wymienionych w punkcie 3:
    a. **Shell Sort z algorytmem bazowym Insertion Sort oraz przyrostami Knutha**:

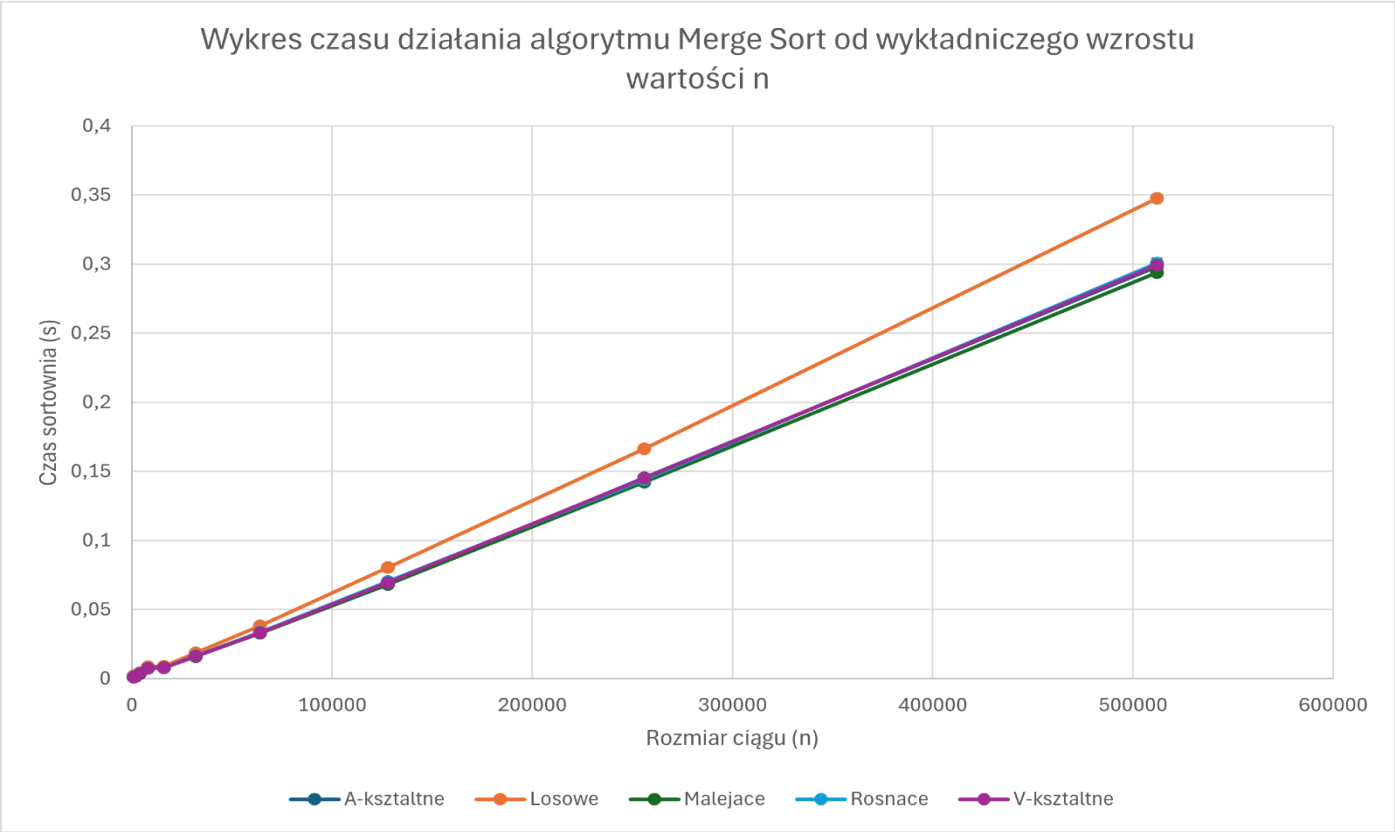




Algorytm Shell Sort pokazuje dużą odporność na wprowadzany rodzaj danych wejściowych. Zarówno dla danych losowych, jak i specyficznych takich jak np. A-kształtne i V-kształtne, czasy pozostają do siebie zbliżone. Choć jest ulepszoną wersją sortowania przez wstawianie, to jego złożoność pozwala na efektywne sortowanie przy wzroście liniowym jak i wykładniczym.

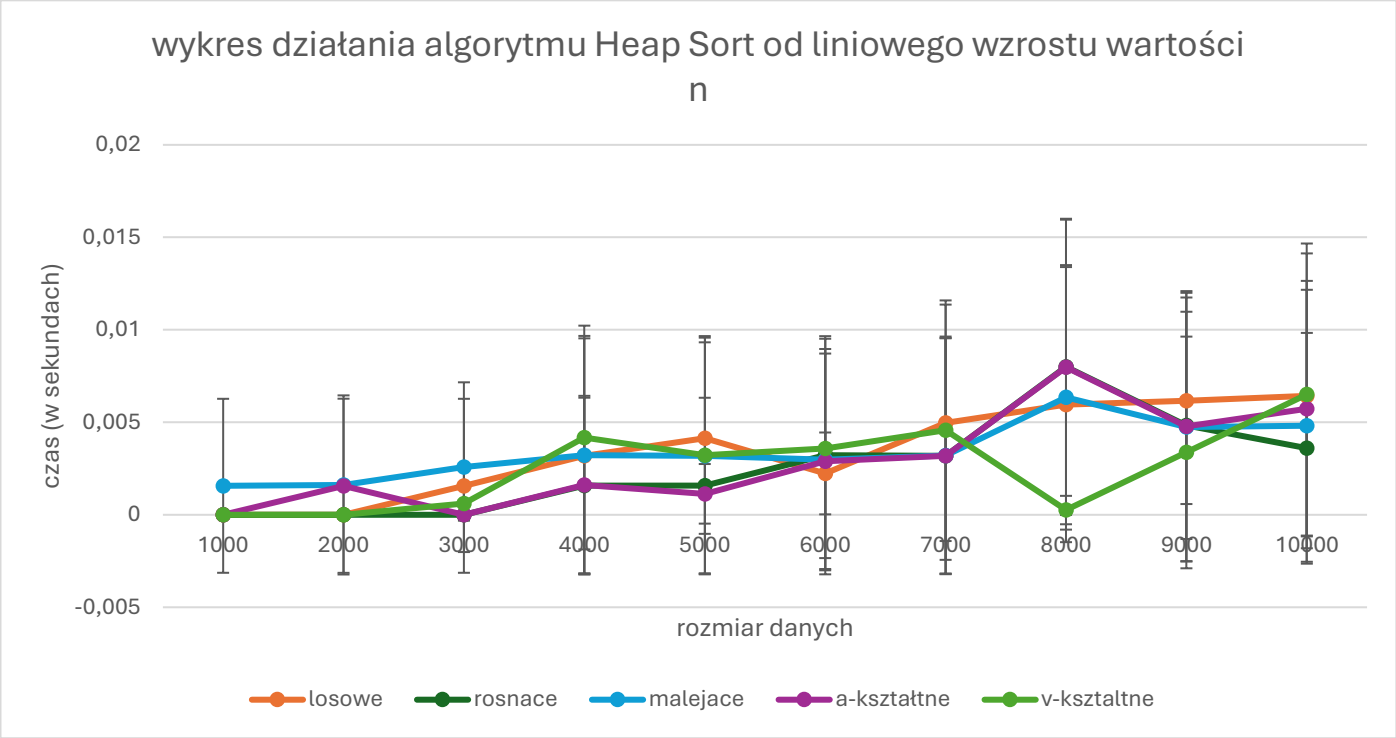
b. **Merge Sort:**

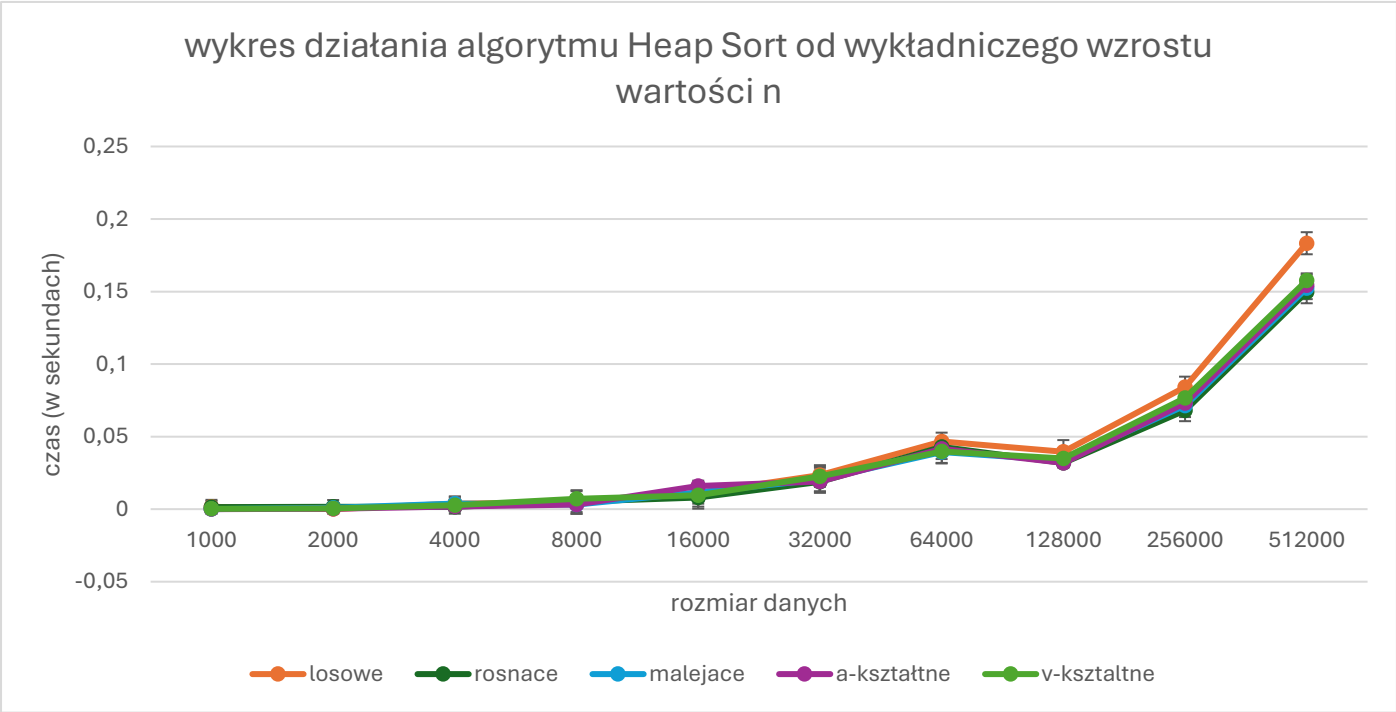




Wykresy stanowią idealny dowód na stabilność algorytmu Merge Sort. Poszarpane krzywe na górnym wykresie (wzrost liniowy) to jedynie efekt szumu systemowego dla bardzo krótkich czasów pomiaru. Rzeczywistą naturę algorytmu ukazuje wykres dolny (wzrost wykładniczy) – niezależnie od początkowego ułożenia danych, wszystkie linie rosną w tym samym, przewidywalnym tempie. Potwierdza to niezmienną złożoność O(nlogn) oraz całkowitą odporność Merge Sorta na przypadek pesymistyczny.

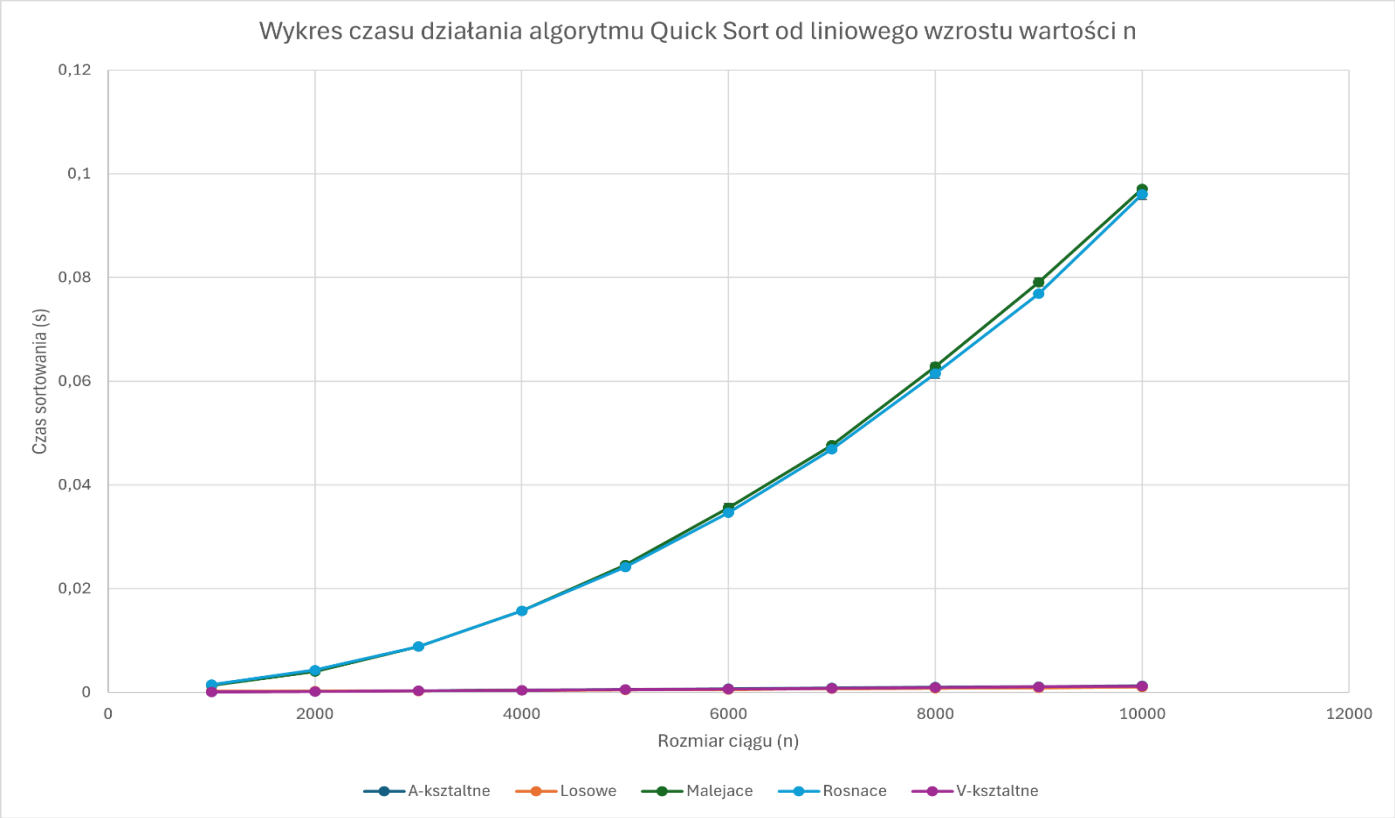
c. **Heap Sort:**

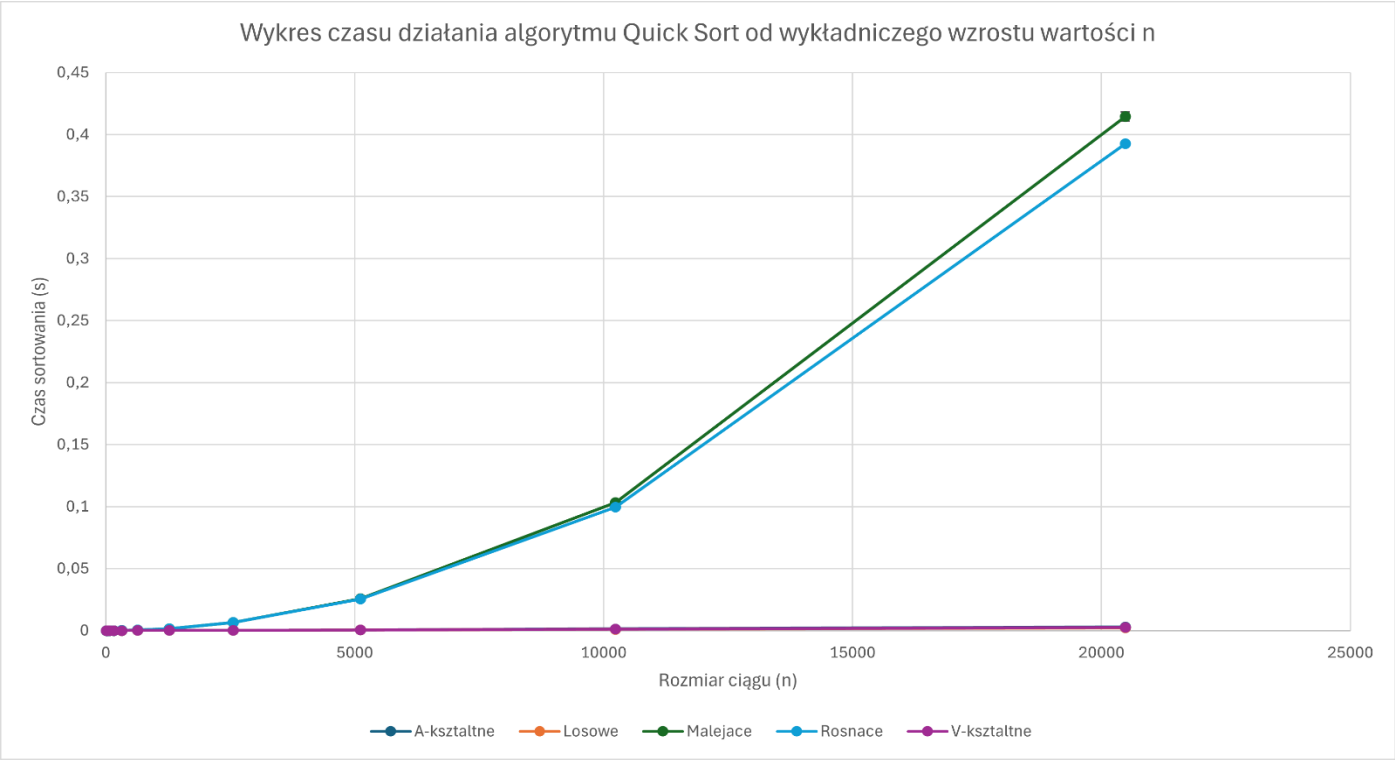




Heap Sort pokazuje stałą wydajność czasową rzędu O(n logn) niezależnie od rodzaju danych. Na wykresach nie ma jakiś gwałtownych skoków czasu dla przypadków pesymistycznych. Zarówno dla danych liniowych jak i wykładniczych czasy nie odbiegają za bardzo do siebie. Ten algorytm jest też bardzo bezpieczny względem pamięci przez używanie sortowania w miejscu.

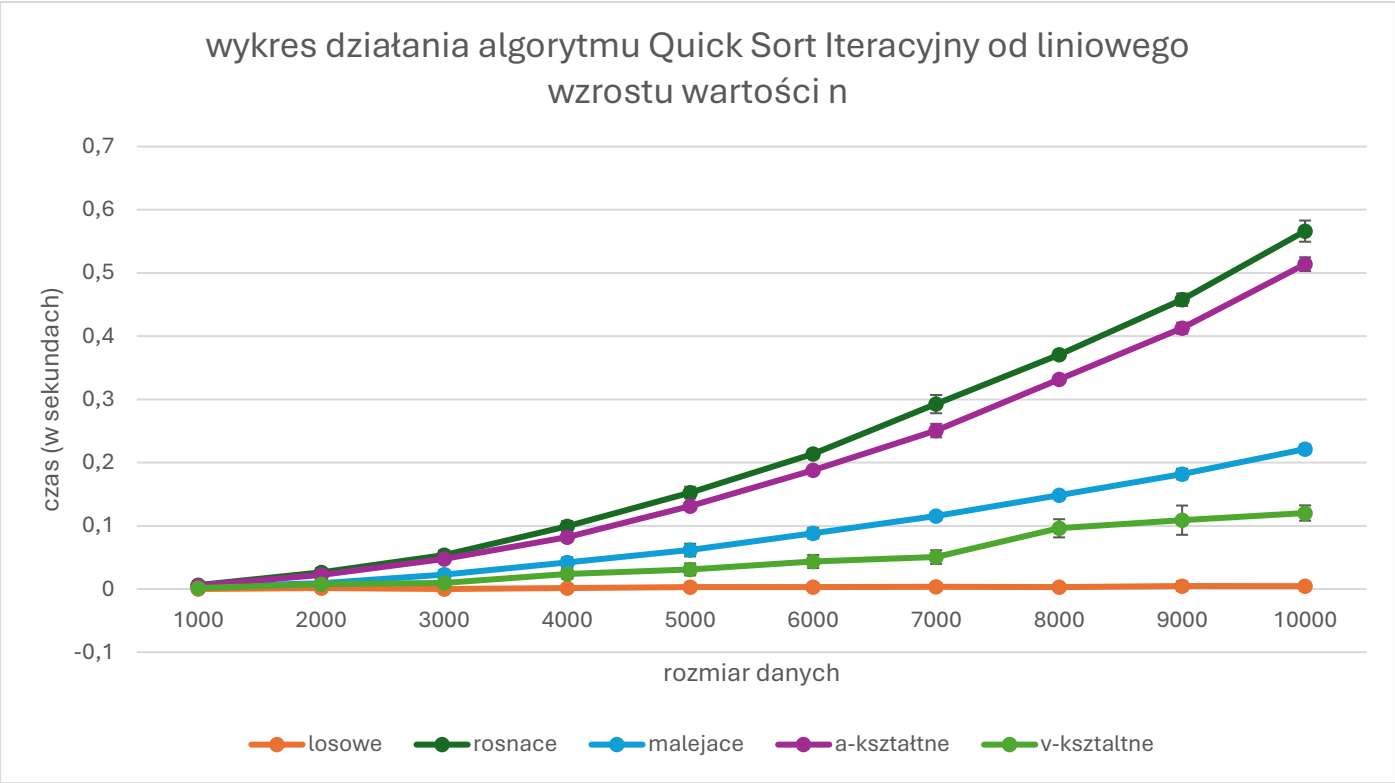
d. **Quick Sort Rekurencyjny z pivotem, którym jest pierwszy element ciągu**:

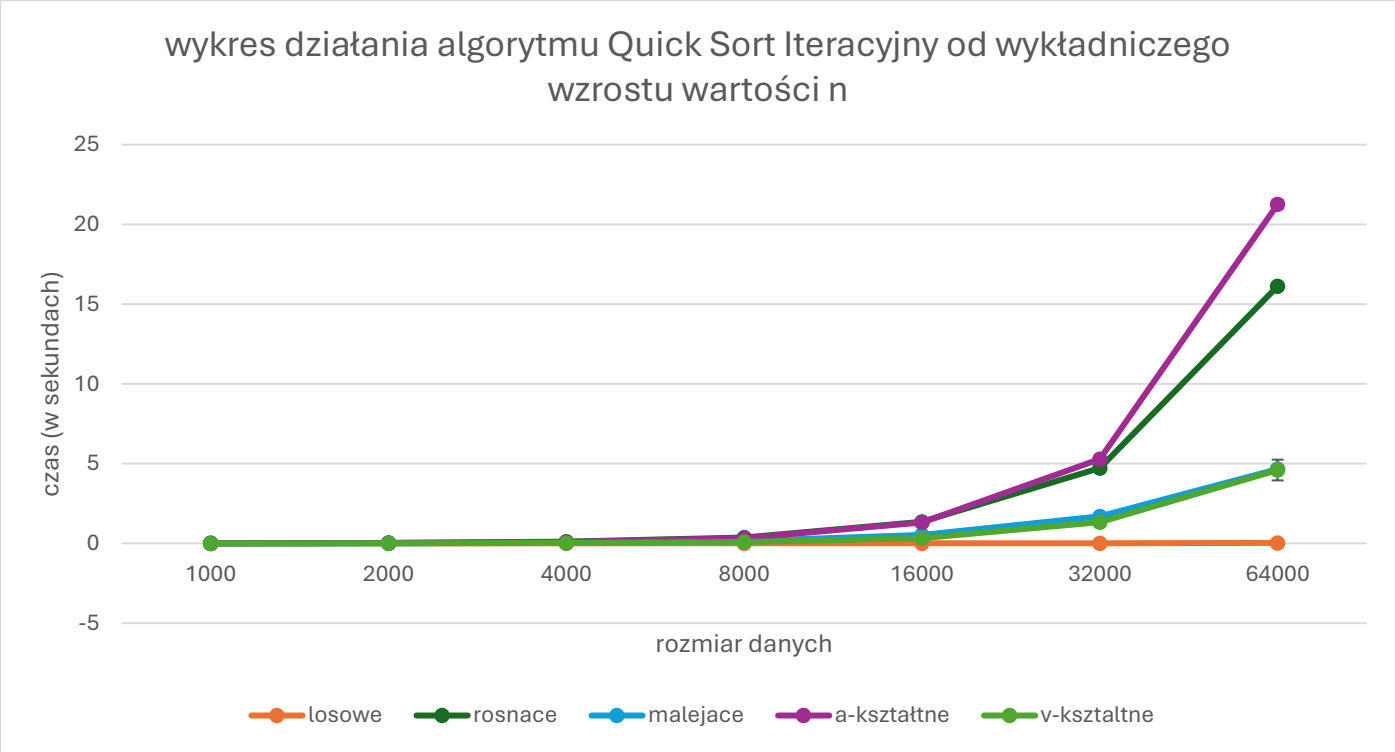




Wykresy pokazują skrajną wrażliwość algorytmu na ułożenie danych. Dla ciągów już posortowanych uderza on w pesymistyczną złożoność O(n^2) (strome parabole na wykresach), podczas gdy dla pozostałych działa w optymalnym czasie O(n log n) (płaskie linie przy osi X). **Testy celowo przerwano przy n=20480**, aby głęboka rekurencja w przypadku pesymistycznym nie wywołała błędu przepełnienia stosu systemowego (**Stack Overflow**).

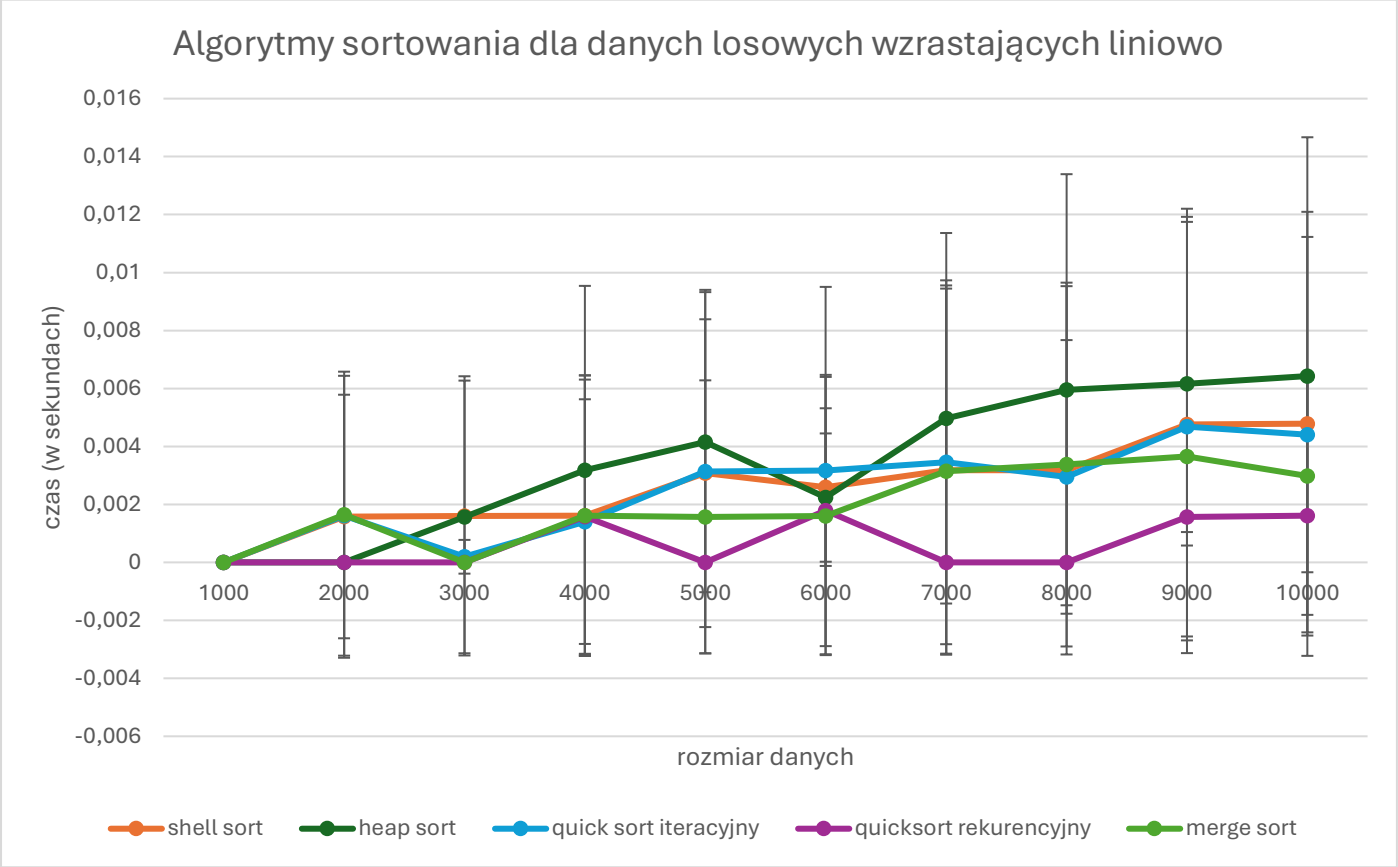
e. **Quick Sort Iteracyjny z pivotem, którym jest pierwszy element ciągu**:

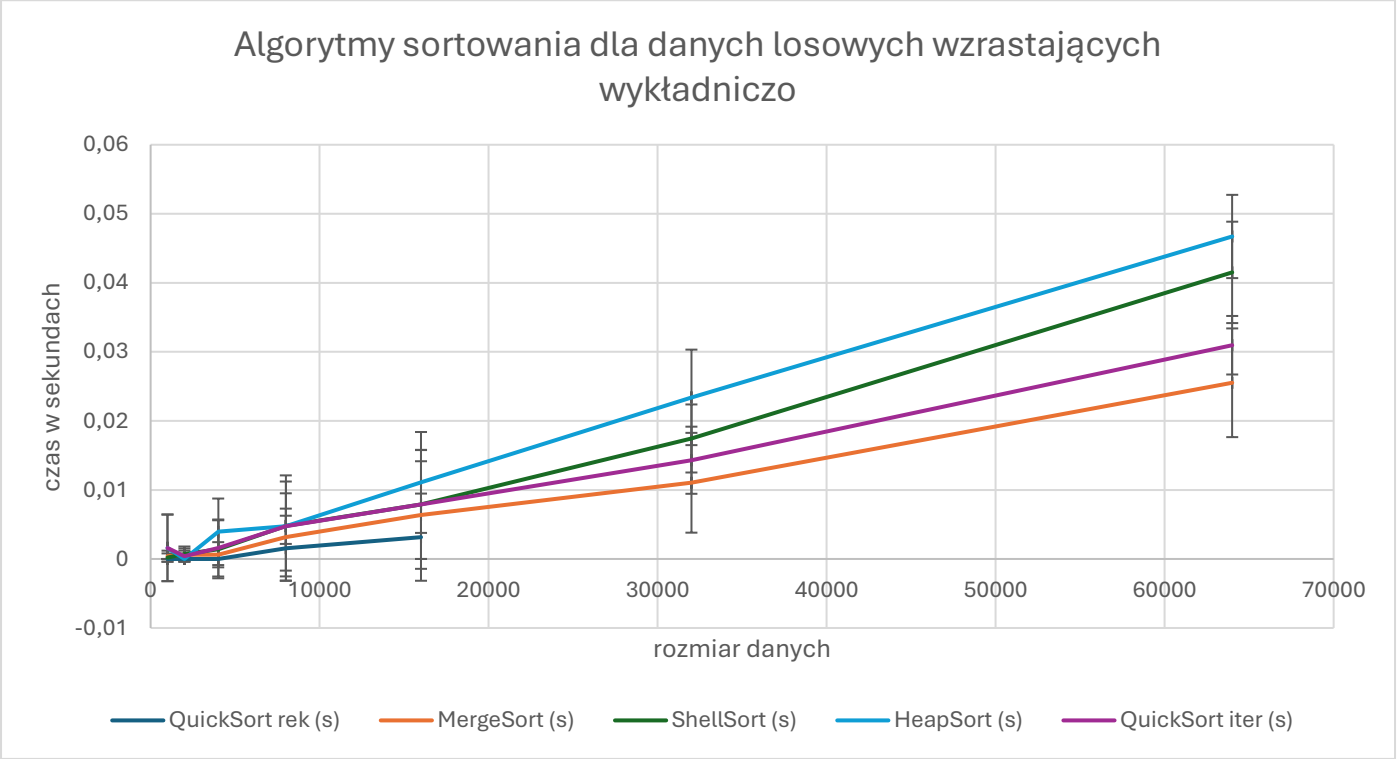




Quick Sort Iteracyjny w przeciwieństwie do Quick Sorta Rekurencyjnego uniknął przepełnienia stosu dzięki wykorzystaniu jawnego stosu, co pozwala na wykonanie pomiarów dla większej ilości danych. Niemniej jednak, kształt wykresu wskazuje na pesymistyczną złożoność obliczeniową $O(n^2)$ wynikającą z wyboru pivota (pierwszy wyraz). Jednak dla danych losowych pozostaje jednym z najszybszych algorytmów.

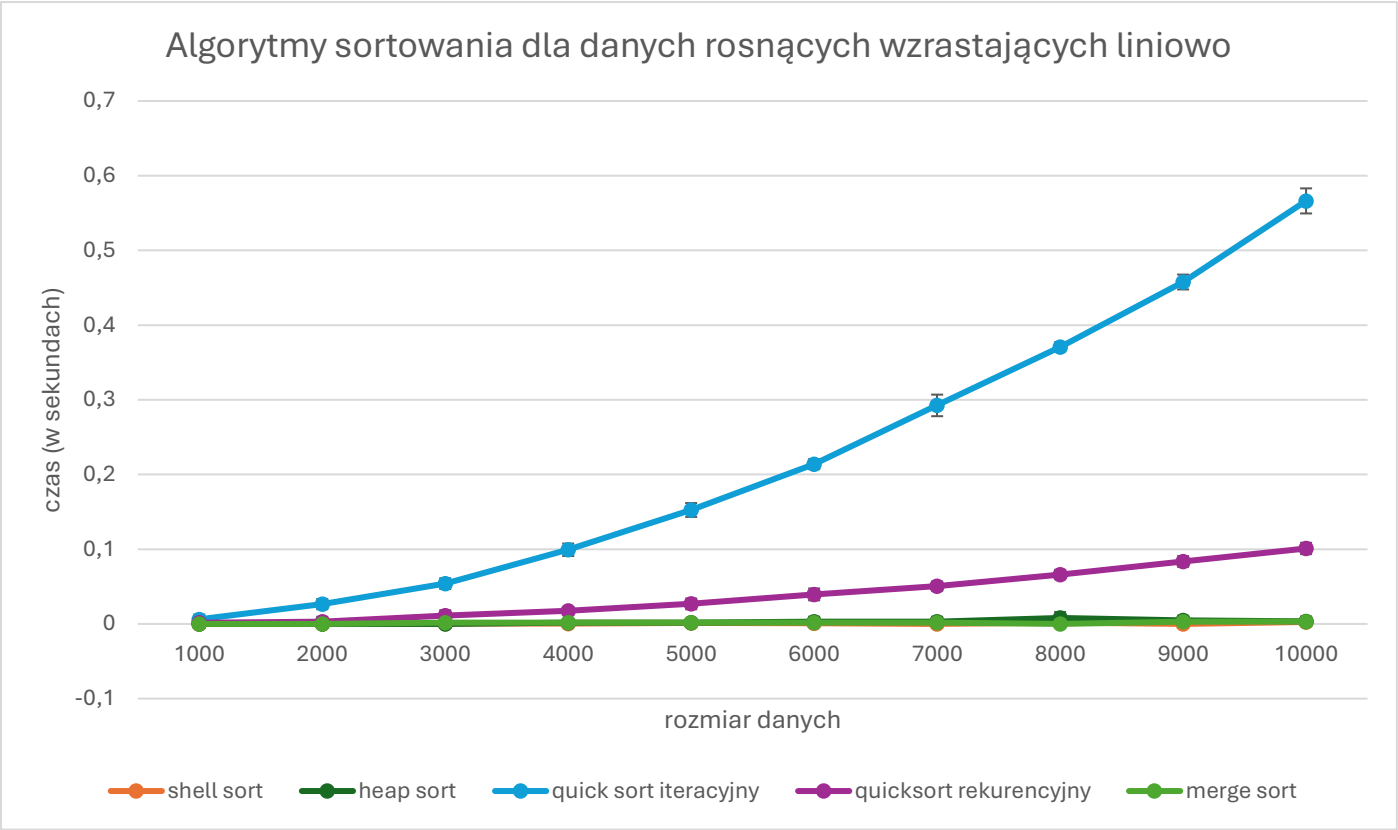
5. **Analiza wyników** – porównanie algorytmów względem typu danych, który jest najkorzystniejszy:
   a. **Dla danych losowych**:

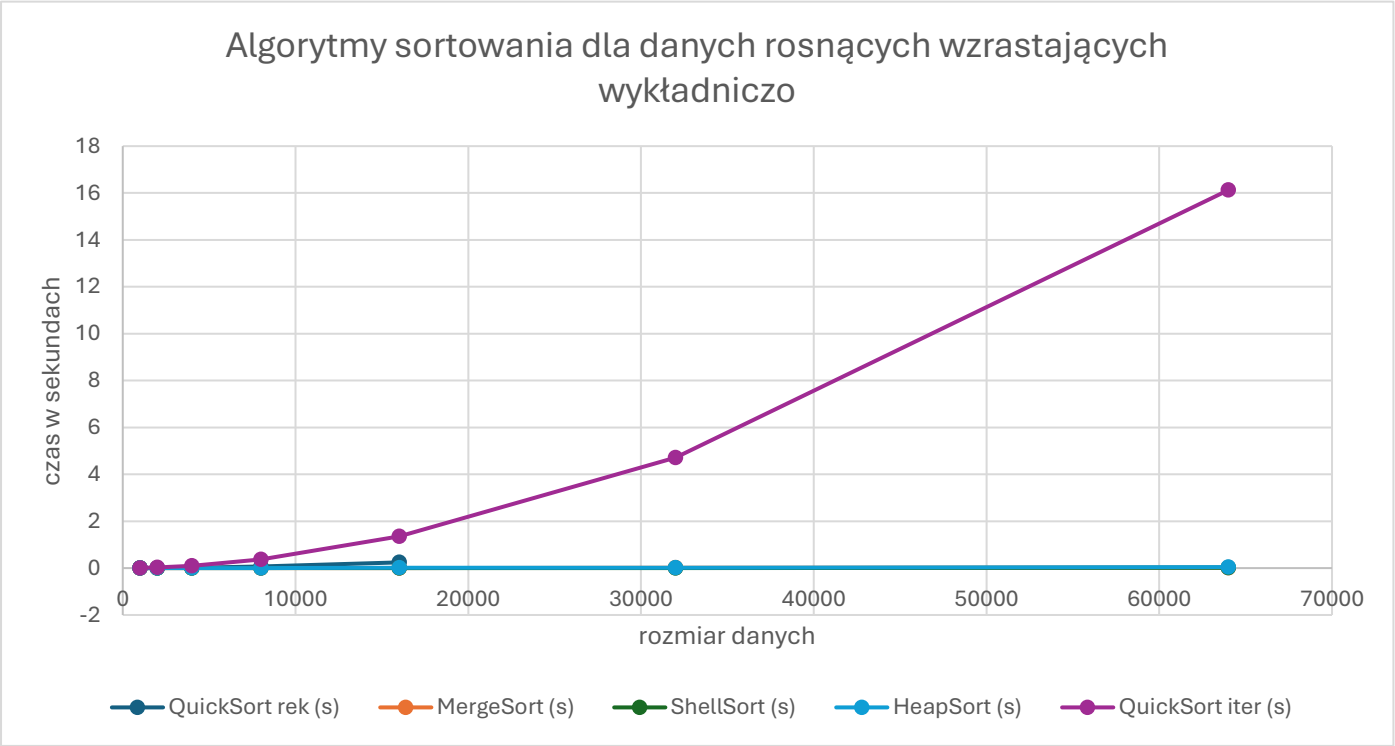




Dla danych losowych wszystkie algorytmy mają w miarę porównywalne czasy, jednak najszybszym z nich pozostaje quick sort zarówno rekurencyjny jak i iteracyjny, chociaż jak widać na wykresie dla danych rosnących wykładniczo quick sort rekurencyjny przestaje działać (stack overflow). Megre Sort, Heap Sort i Shell Sort wykazują bardzo zbliżone czasy, zatem są bardzo dobrą stabilną alternatywą.

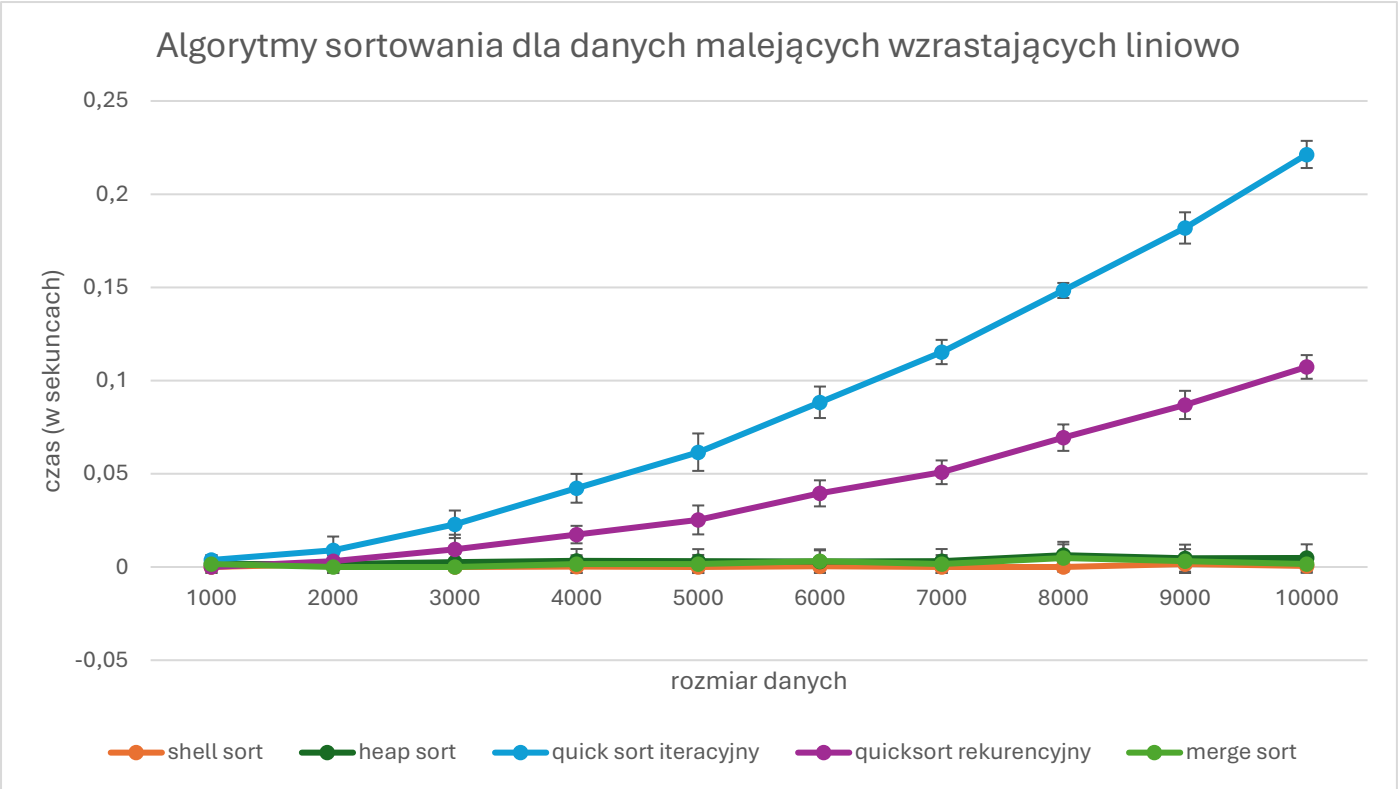
b. **Dla danych rosnących**:

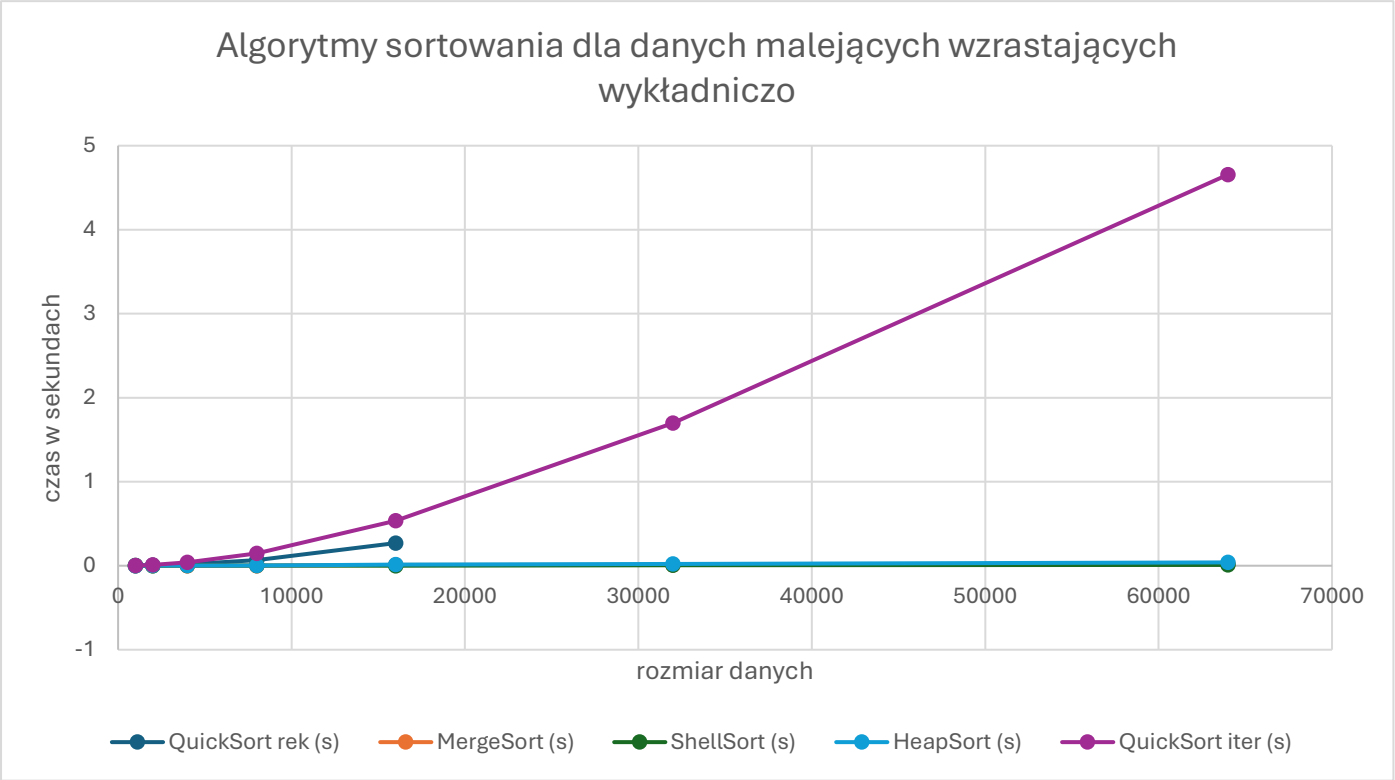




Zestawienie dla danych rosnących najlepiej pokazuje różnice w klasach złożóności obliczeniowej. Shell Sort, Merge Sort oraz Heap Sort działają niezwykle efektywne, niemal w czasie liniowym (merge Sort nadal ma O(n log n)). Jednak na wykresie widzimy tutaj „wystrzał" krzywej w górę co oznacza przejście w złożoność $O(n^2)$ dla algorytmu Quick Sorta zarówno rekurencyjnego jak i iteracyjnego (wersja z pivotem na początku).

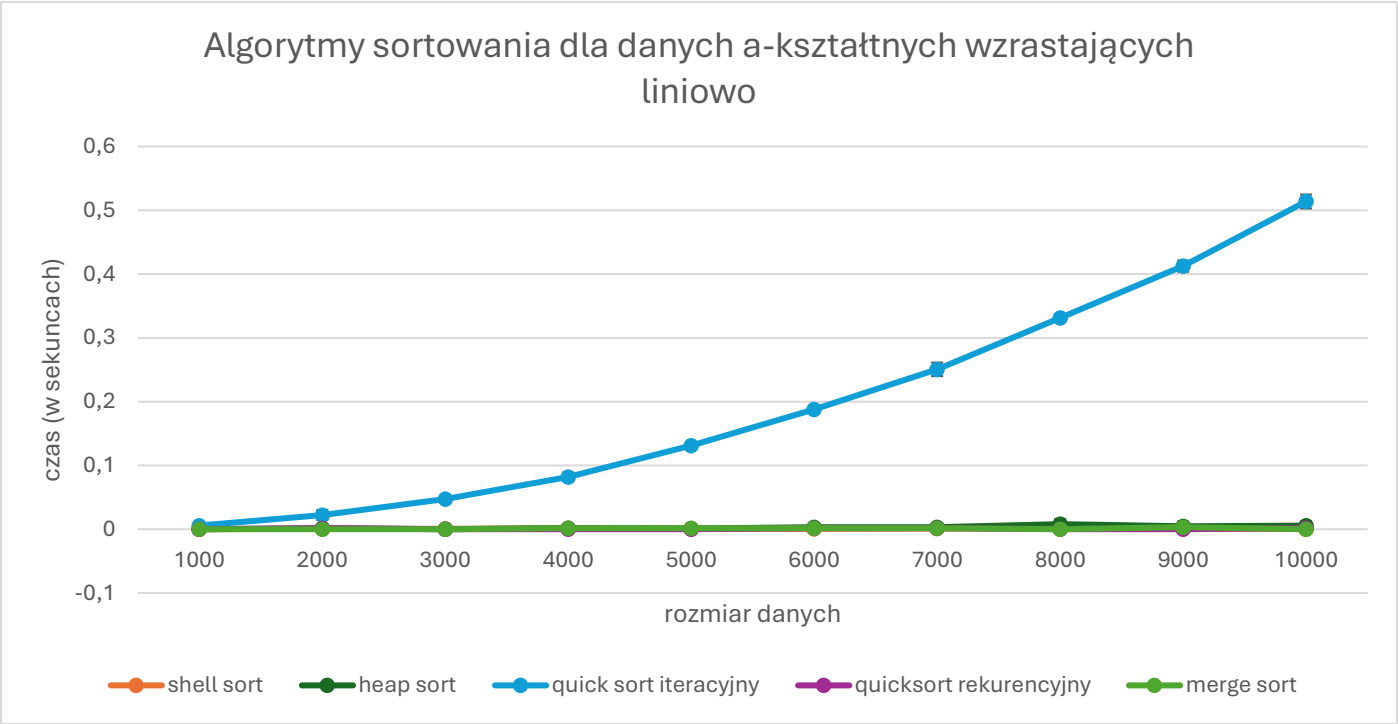
c. **Dla danych malejących:**

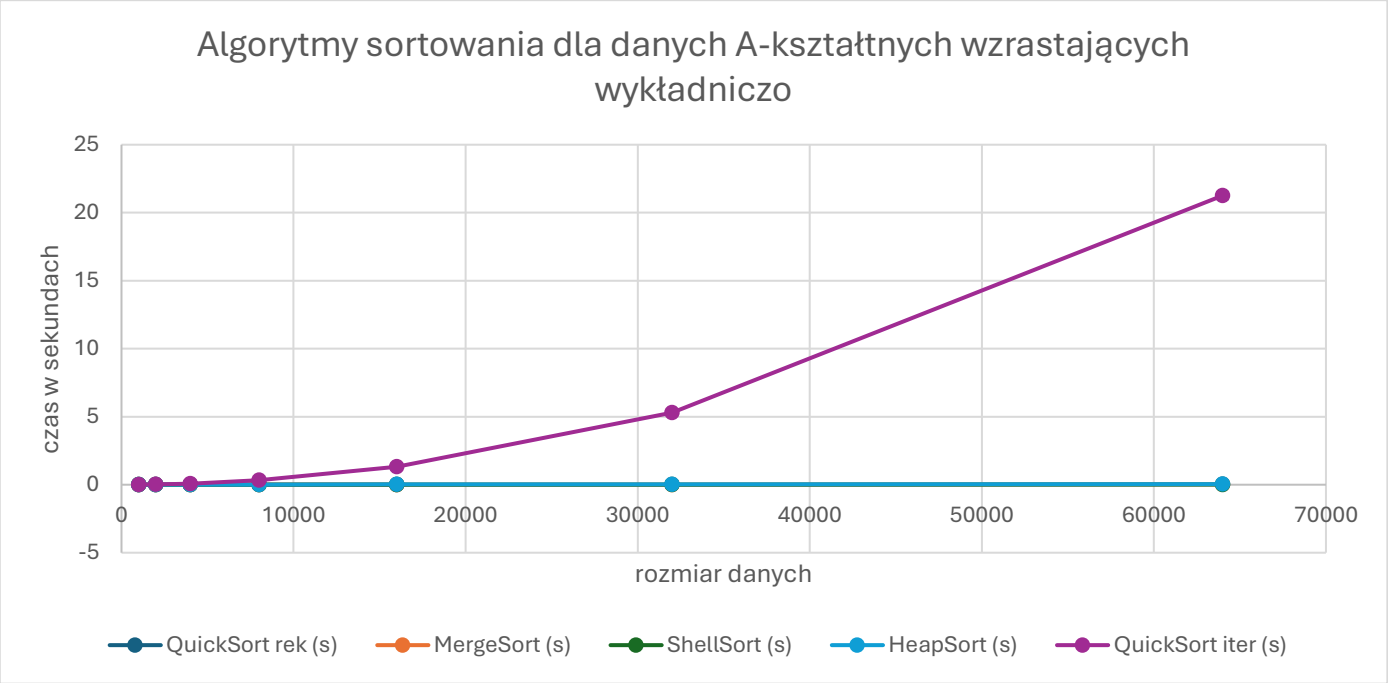




Dla danych malejących mamy sytuację analogiczną do danych rosnących: ponownie Heap Sort, Merge Sort oraz Shell sort wykazują całkowitą odporność na kierunek sortowania danych, utrzymując stabilne czasy. Krzywa dla Quick Sorta jednak znowu „wystrzeliwuje” do góry, co oznacza że ten algorytm nadal drastycznie zwalnia do poziomu $O(n^2)$. Wybór pivota jako pierwszego elementu dla ciągu malejącego jest tak samo nieefektywny co przy danych dla ciągu rosnącego.

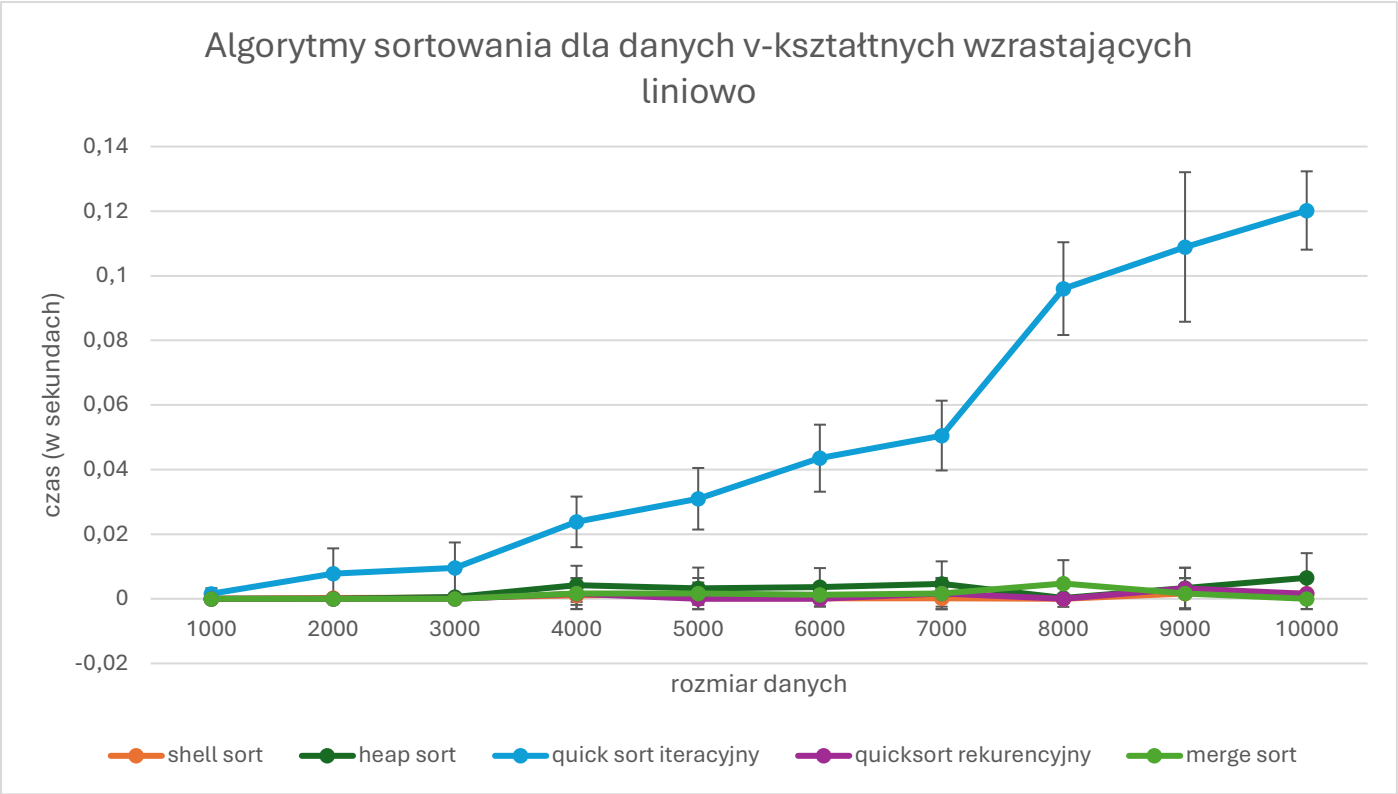
d. **Dla danych A-kształtnych**:

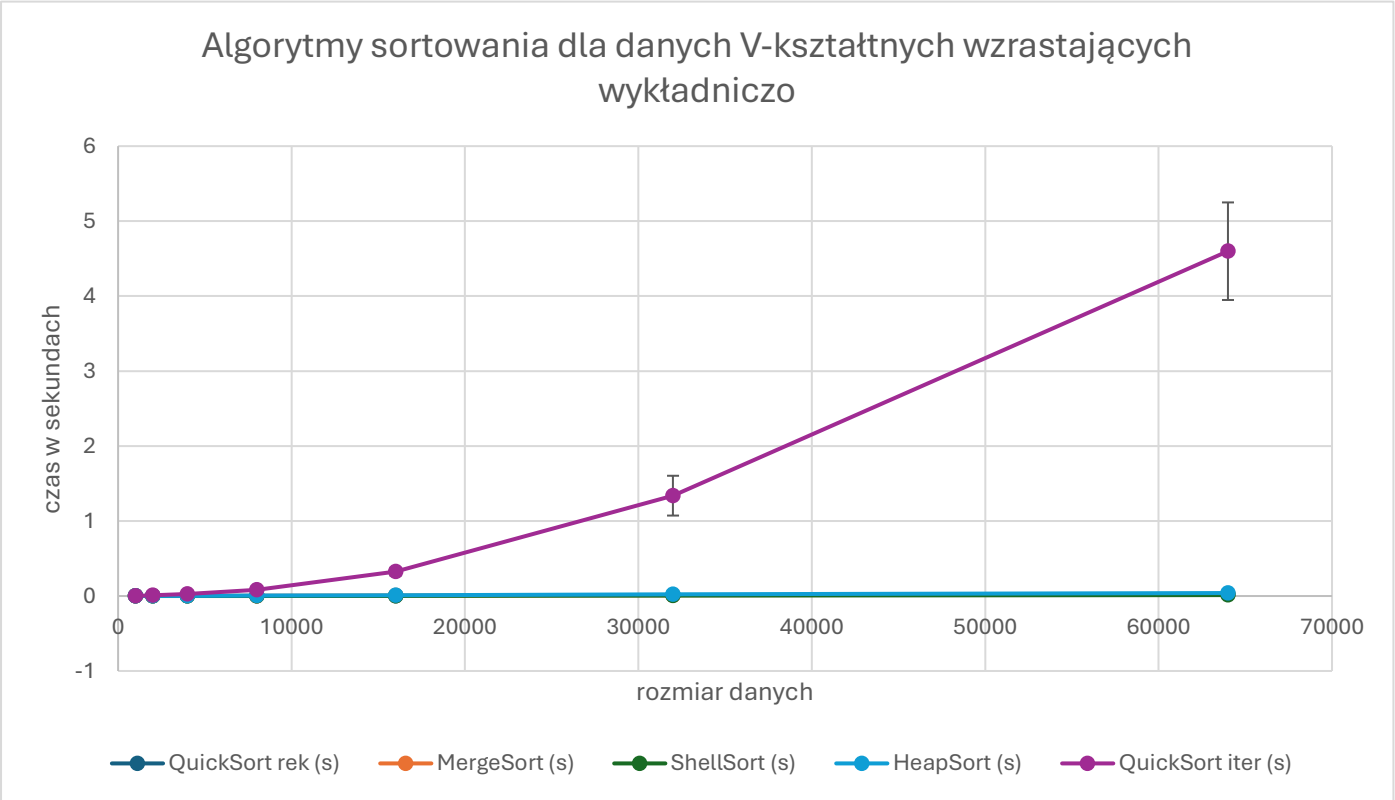




Dla danych A-kształtnych, które charakteryzują się tym, że połowa zbioru jest posortowana rosnąco, a druga malejąco liderami stabilności czasowej nadal jest Merge Sort, Heap Sort oraz Shell Sort. Te algorytmy mają niemal identyczne czasy jak dla danych losowych. Potwierdza to ich odporność na specyficzny układ elementów. Ponownie na wykresach mamy stromą parabolę dla Quick Sorta Iteracyjnego (złożoność $O(n^2)$), ale za to Quick Sort Rekurencyjny utrzymuje się bardzo nisko co sugruje wydajność bliską O(n logn). Wynika to z faktu, że mimo pivota jako pierwszego elementu podziały rekurencyjne nie stały się skrajnie asymetryczne na tyle wcześnie, by spowolnić algorytm przed osiągnięciem limitu n.

e. **Dla danych V-kształtnych**:





Przypadek danych, gdzie pierwsza połowa danych jest posortowana malejąco, a druga rosnąco. Przewidywalnie Merge Sort, Heap Sort oraz Shell Sort oferują najbardziej przewidywalną i stabilną pracę. Wykresy tych algorytmów przy wzroście wykładniczym są niemal liniowe, co jest cechą idealnego O(n logn). Podobnie jak u danych A-kształtnych, Quick Sort Iteracyjny wykazuje złożoność $O(n^2)$ i jest wolniejszy od wersji rekurencyjnej, która wykazała się dużą szybkością.

Tylko na niektórych powyższych wykresach widać wyraźnie odchylenie standardowe, gdyż w wielu przypadkach było ono na tyle małe, iż nie jest ono zauważalne. Dla pewności wszystkie wartości są wypisane w tabelkach na samym końcu sprawozdania.

6. **Analiza porównawcza obu eksperymentów**.

Eksperyment 1 (wzrost liniowy w przedziale od 1000 do 10 000 elementów) ukazał działanie algorytmów w skali "mikro". W tym środowisku wszystkie testowane metody (Quick Sort rekurencyjny oraz iteracyjny, Merge Sort, Shell Sort, Heap Sort) radziły sobie z podobną szybkością, a czasy sortowania wynosiły ułamki sekund. Wyniki uzyskane w tym eksperymencie były silnie podatne na szum systemu operacyjnego (fluktuacje na wykresach), co utrudniało jednoznaczną ocenę klasy złożoności.

Z kolei Eksperyment 2 (wzrost wykładniczy, sięgający w niektórych przypadkach do 512000 elementów) zupełnie zmienił perspektywę. Wyniki bezpośrednio uwypukliły fizyczne ograniczenia metod rekurencyjnych – Quick Sort, który w eksperymencie liniowym dla danych posortowanych działał bezbłędnie, w skali wykładniczej natychmiast uderzył w barierę sprzętową (przepełnienie stosu systemowego przy $n > 20480$), co wymusiło przerwanie testów. Równocześnie eksperyment ten udowodnił potęgę stabilnych algorytmów o złożoności $O(n \log n)$, takich jak Merge Sort, które nawet dla pół miliona elementów utrzymały płaski i stabilny czas wykonania. Eksperyment liniowy pokazał zatem zachowanie programów w warunkach optymalnych, podczas gdy eksperyment wykładniczy pokazał ich matematyczną naturę i granice wydajności sprzętowej.

W którym eksperymencie łatwiej zaobserwować różnice między algorytmami?

Zdecydowanie w eksperymencie 2 (ze wzrostem wykładniczym). Przy wzroście liniowym (od 1000 do 10000 elementów) czasy wykonania dla szybkich algorytmów są wciąż mikroskopijne (rzędu pojedynczych milisekund), przez co krzywe na wykresach często się na siebie nakładają lub są zaburzone. Dopiero skok wykładniczy do kilkudziesięciu tysięcy elementów weryfikuje wydajność poszczególnych metod sortowania, uwidaczniając drastyczne różnice w czasie ich działania.

Jak zmienia się kształt wykresów przy wzroście wykładniczym?

Wykresy wyraźnie się "rozjeżdżają" w zależności od złożoności algorytmu i typu danych. Dla algorytmów stabilnych o złożoności $O(n \log n)$, jak np. Merge Sort krzywa rośnie stosunkowo łagodnie i przewidywalnie. Z kolei dla przypadków pesymistycznych (np. Quick Sort dla danych posortowanych ze skrajnym pivotem), krzywa wystrzeliwuje niemal pionowo w górę, przyjmując ostry kształt rosnącej paraboli charakterystycznej dla złożoności kwadratowej $O(n^2)$.

Czy wzrost wykładniczy lepiej ujawnia charakter złożoności obliczeniowej?

Tak. Wzrost wykładniczy pozwala zaobserwować tzw. asymptotyczne tempo wzrostu funkcji. Przy małych wartościach n o czasie działania decydują narzuty sprzętowe, czas wywołania funkcji czy zarządzanie pamięcią. Dopiero przy potężnych, rosnących wykładniczo zbiorach danych te stałe wartości tracą na znaczeniu, a na pierwszy plan wysuwa się matematyczna złożoność algorytmu.

Czy dla małych wartości n różnice między algorytmami są istotne?

Różnice te są całkowicie nieistotne w praktyce. Dla małych tablic (np. $n < 4000$) badane algorytmy kończą pracę w ułamkach milisekund. Niewielkie różnice między nimi giną w tzw. "szumie systemu operacyjnego" (przerwania sprzętowe, procesy w tle działające na komputerze). Sprawia to, że dla małych zbiorów danych wybór między zaawansowanymi algorytmami nie przynosi zauważalnych korzyści czasowych.

Jak zmienia się odchylenie standardowe wraz ze wzrostem n?

Wartość bezwzględna odchylenia standardowego rośnie wraz ze wzrostem wielkości zbioru n. Wynika to z faktu, że im dłużej algorytm pracuje nad ogromną tablicą, tym większe jest prawdopodobieństwo, że system operacyjny zabierze na chwilę zasoby procesora na inne zadania. Powoduje to większe wahania i "rozrzut" czasu wykonania między poszczególnymi próbami dla tego samego rozmiaru n.

7. **Analiza złożóności obliczeniowej**:

Poniższa tabela przedstawia teoretyczne klasy złożoności czasowej dla badanych algorytmów:

Algorytm	Przypadek optymistyczny	Przypadek średni	Przypadek pesymistyczny
Shell Sort	O(n logn)	$O(n^{1.25})$	$O(n^{1.5})$
Megre Sort	O(n logn)	O(n logn)	O(n logn)
Heap Sort	O(n logn)	O(n logn)	O(n logn)
Quick Sort Rekurencyjny	O(n logn)	O(n logn)	$O(n^2)$
Quick Sort Iteracyjny	O(n logn)	O(n logn)	$O(n^2)$

Związek między teorią, a liczbą operacji – złożoność teoretyczna określa asymptotyczne tempo wzrostu liczby dominujących operacji wraz ze wzrostem rozmiaru danych n. W algorytmach o złożóności O(n logn) 10-krotny wzrost danych powoduje wzrost czasu nieco ponad 10-krotny, a w przypadku złożóności kwadratowej $O(n^2)$, 10-krotny wzrost danych skutkuje aż 100-krotnym wydłużeniem czasu pracy algorytmu.

W większości przypadków wyniki eksperymentalne w pełni potwierdzają teorię, co najlepiej widać w eksperymentach ze wzrostem wykładniczym. Merge Sort i Heap Sort wykazały niemal identyczne czasy niezależnie od typu danych, co potwierdza ich stabilną złożóność O(n logn) niezależną od ułożenia danych. Również duża odporność na rodzaj danych wykazał algorytm Shell Sort urzymując czasy zbliżone do algorytmów klasy O(n logn). Za to badania algorytmu Quick Sort potwierdziły jego „dwubiegunowość" względem danych. Dla losowych był najszybszy, jednak dla danych posortowanych rósł parabolicznie co udowadnia jego wydajność $O(n^2)$. Również Quick Sort Rekurencyjny przy dużej ilości danych posortowanych, głębokość rekurencji rośnie liniowo do doprowadziło do Stack Overflow (w badaniu dla n > 20480). A Quick Sort Iteracyjny pozwolił na posortowanie większej ilości danych, nawet dla przypadków pesymistycznych, ale mimo wszystko wykazuje złożóność $O(n^2)$ dla danych posortowanych.

Przypadki występowanie odchyleń:

- **Dla małych zbiorów danych (n < 4000)** – różnice czasowe między algorytmami w praktyce są nieistotne, ponieważ zadania kończą się zwykle w ułamkach milisekund
- **Dla danych uporządkowanych (malejące/rosnące)** – algorytm Quick Sorta wykazuje skrajną wrażliwość na ułożenie danych, co przy wyborze skrajnego pivota powoduje „wystrzał" czasu w górę i przejście w złożoność $O(n^2)$.
- **Skala wykładnicza a liniowa** – wzrost liniowy (do 10 000 elementów) ukazuje działanie algorytmu w małej skali, gdzie różnice czasowe są zacierane przez czynniki zewnętrzne. Dopiero skoki wykładnicze pokazują ich prawdziwą naturę matematyczną
- **Błąd przepełnienia stosu dla Quick Sorta Rekurencyjnego** – przy danych posortowanych (n > 20480) następuje przerwanie testów z powodu bariery sprzętowej

Przyczyny odchyleń:

- **Szum systemu operacyjnego** – współczesne komputery wykonują wiele zadań jednocześnie. Przerwania sprzętowe, procesy działające w tle oraz zarządzanie zasobami przez system operacyjny wprowadzają fluktuacje, które są szczególnie widoczne przy bardzo krótkich czasach pomiaru (szum systemowy).
- **Architektura sprzętowa i limity pamięci**

- **Stos systemowy** – Quick Sort rekurencyjny opiera się na fizycznym stosie, co przy głębokiej rekurencji (dane posortowane) prowadzi do awarii programu, mimo że teoretycznie algorytm powinien kontynuować pracę.
- **Zasoby procesora** – Im dłużej trwa sortowanie ogromnej tablicy, tym większa szansa, że system operacyjny na chwilę przydzieli moc procesora innemu zadaniu, co zwiększa odchylenie standardowe wraz ze wzrostem n.

- **Implementacja a definicja matematyczna**
  - **Wybór pivota** – Teoretyczna średnia złożoność Quick Sorta to O(n logn), ale wybór pierwszego elementu jako pivota celowo wywołuje przypadek pesymistyczny dla danych posortowanych.
  - **Narzut iteracji** – Quick Sort iteracyjny unika błędu stosu, ale jest marginalnie wolniejszy w przypadkach optymalnych ze względu na konieczność ręcznego zarządzania strukturą danych (jawny stos).

8. **Wnioski**:

Najbardziej efektywny algorytm w praktyce: Biorąc pod uwagę stabilność i niezawodność, zwycięzcą jest **Merge Sort**. Niezależnie od ułożenia danych wejściowych (losowe, rosnące, malejące, A-kształtne, V-kształtne), zawsze gwarantował on optymalny czas wykonania rzędu O(n log n). Jeśli jednak zależy nam na czystej szybkości działania i mamy pewność, że dane są silnie zróżnicowane (losowe), najszybszym wyborem w praktyce pozostaje algorytm Quick Sort.

Złoty środek pamięciowy (Heap Sort): Sortowanie przez kopcowanie okazało się najbezpieczniejszym i najbardziej uniwersalnym wyborem pod kątem zarządzania pamięcią. W przeciwieństwie do algorytmu Merge Sort, Heap Sort sortuje dane w miejscu (złożoność pamięciowa O(1) i nie wymaga alokowania potężnych tablic pomocniczych. Jednocześnie, w przeciwieństwie do Quick Sorta, jest całkowicie odporny na przypadek pesymistyczny, gwarantując zawsze czas O(n log n).

Algorytm najbardziej wrażliwy na postać danych wejściowych: Zdecydowanie jest to **Quick Sort**. Przeprowadzony eksperyment udowodnił, że o ile dla danych losowych algorytm ten deklasuje konkurencję, o tyle podanie mu ciągów rosnących lub malejących paraliżuje jego działanie. Złożoność czasowa drastycznie spada wtedy z optymalnego O(n log n) do pesymistycznego $O(n^2)$, co na wykresach objawia się krzywą w kształcie paraboli.

Porównanie Quick Sort rekurencyjny vs iteracyjny: Wersja rekurencyjna opiera się na fizycznym stosie systemu operacyjnego, co jest jej największą wadą w przypadku pesymistycznym (głęboka rekurencja dla ciągów posortowanych prowadzi do błędu przepełnienia stosu (stack overflow), co wymusiło przerwanie testów już przy rozmiarze n=20480. Wersja iteracyjna rozwiązuje ten problem, symulując stos w pamięci sterty (Heap). Jest znacznie bezpieczniejsza i pozwala przeliczyć o wiele większe zbiory danych, jednak narzut na zarządzanie własną strukturą danych czyni ją marginalnie wolniejszą dla przypadków optymalnych.

Wpływ wyboru pivota na wyniki: Eksperyment bezpośrednio udowodnił, że wybór skrajnego (np. pierwszego) elementu na pivota jest fatalny w skutkach dla danych częściowo lub całkowicie posortowanych – redukuje to podziały tablicy do pojedynczych elementów i wywołuje przypadek pesymistyczny. Zastosowanie pivota środkowego (wyznaczanego np. jako mediana z trzech elementów) lub wybieranego losowo, z dużym prawdopodobieństwem całkowicie zniwelowałoby ten problem, przywracając złożoność O(n log n) dla każdej postaci ciągu.

Wpływ sposobu skalowania danych: Skalowanie **wykładnicze** bezsprzecznie lepiej oddaje rzeczywiste różnice między algorytmami. W przeciwieństwie do skalowania liniowego (gdzie dla małych wartości n różnice giną w szumie przerwań systemu operacyjnego), skalowanie wykładnicze neutralizuje zakłócenia wynikające z potężnej szybkości współczesnych procesorów. Pozwala to zaobserwować prawdziwy, asymptotyczny charakter funkcji matematycznej opisującej dany algorytm i bezlitośnie obnaża limity sprzętowe testowanych rozwiązań.

TABELE POMIARÓW

ALGORYTM SHELL SORT dla danych wzrastających:

- Liniowo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0	0	0	0	0
2000	0,00158306	0	0,00157186	0,00037559	0,00025963
3000	0,00160548	0,00061018	0	0,00051377	0,00035426
4000	0,00161203	0,00057562	0,00020081	0,0014559	0,00105955
5000	0,00307759	0,00116898	0	0,00082012	0,0015698
6000	0,0025974	0,00078196	0,00030982	0,00053484	0,00020091
7000	0,00318531	0	0	0,00134842	0,00017398
8000	0,00317657	0,00161433	0	0,00174655	0
9000	0,00475493	0	0,00160152	0,00158838	0,00159743
10000	0,00478484	0,00261208	0,00060309	0	0,00161118

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0	0	0	0	0
2000	0,00420174	0	0,00471558	0,00075349	0,00053285
3000	0,00481644	0,00074734	0	0,00087709	0,00071613
4000	0,00483609	0,0008819	0,00060243	0,00288694	0,00200476
5000	0,00530982	0,00350694	0	0,00137236	0,00346168
6000	0,00271973	0,00177657	0,00061964	0,0011102	0,00060273
7000	0,00637072	0	0	0,00404526	0,00052194
8000	0,00635367	0,00484299	0	0,00403044	0
9000	0,00744631	0	0,00480456	0,00476514	0,00479229
10000	0,00730968	0,00459334	0,00092124	0	0,00483354

- Wykładniczo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00020076	0,00020074	0,00043064	0	0
2000	0,00070183	0,00072796	0	0,00020064	0
4000	0,0014006	0,00040128	0,00019288	0,0007761	0
8000	0,00478972	0,00159185	0,00040103	0,00435733	0
16000	0,0078861	0	0	0,00278665	0,00321204
32000	0,01745259	0,0058425	0,00434702	0,00786495	0,00737836

64000	0,04151335	0,01466472	0,00940995	0,01323638	0,01243365
128000	0,09134869	0,0276556	0,01358525	0,0271023	0,02479423
256000	0,21170973	0,06355493	0,03535635	0,0533601	0,02330205
512000	0,45745131	0,11960532	0,07594089	0,10969567	0,04017344

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00060228	0,00060222	0,00129192	0	0
2000	0,00110302	0,00218388	0	0,00060192	0
4000	0,0042018	0,00080256	0,00057864	0,00095323	0
8000	0,00731645	0,0035283	0,00080206	0,00672245	0
16000	0,00788629	0	0	0,0028703	0,00642409
32000	0,00492872	0,00657942	0,00600011	0,00786513	0,00595947
64000	0,007344	0,00190527	0,00598668	0,00555844	0,00645503
128000	0,00492031	0,00340153	0,00315829	0,00496041	0,00683468
256000	0,01275055	0,00695101	0,00433881	0,0038371	0,01071943
512000	0,01996552	0,00472981	0,01113391	0,0082929	0,00846707

ALGORYTM MERGE SORT dla danych wzrastających:

- Liniowo:

Średni czas:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000676	0,001496	0,000775	0,000963	0,000598
2000	0,000918	0,001288	0,000981	0,001046	0,000907
3000	0,001358	0,001571	0,001377	0,001384	0,001366
4000	0,001864	0,002093	0,001824	0,00185	0,001871
5000	0,002348	0,002668	0,002336	0,002371	0,00237
6000	0,002832	0,003189	0,0028	0,002795	0,002821
7000	0,003311	0,003789	0,003291	0,003344	0,003313
8000	0,003936	0,004336	0,003775	0,003775	0,003775
9000	0,004322	0,004954	0,004299	0,004304	0,004402
10000	0,004823	0,005512	0,004855	0,004819	0,004877

Odchylenie standardowe:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000027	0,000193	0,000058	0,000084	0,00002
2000	0,000024	0,000079	0,000048	0,000055	0,000054
3000	0,000045	0,000063	0,000062	0,000043	0,000052
4000	0,000059	0,000055	0,000022	0,000073	0,000052
5000	0,000057	0,000054	0,000037	0,000047	0,000034
6000	0,000072	0,000062	0,000071	0,000049	0,00007
7000	0,000055	0,000084	0,000055	0,000053	0,000045
8000	0,000136	0,000063	0,000061	0,000048	0,000047
9000	0,000056	0,000069	0,000066	0,000073	0,000055
10000	0,000076	0,000079	0,000079	0,000106	0,000065

- Wykładniczo:

Średni czas:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000426	0,000474	0,000442	0,000427	0,000436
2000	0,000906	0,001016	0,000899	0,000914	0,000928
4000	0,001852	0,002097	0,001838	0,001853	0,001842
8000	0,003853	0,004319	0,003816	0,003848	0,003799
16000	0,007874	0,008949	0,007931	0,00785	0,007804
32000	0,016095	0,01858	0,016004	0,016547	0,016075
64000	0,033134	0,038317	0,032785	0,033675	0,033307
128000	0,068229	0,080576	0,068273	0,07021	0,06946
256000	0,145318	0,166308	0,142165	0,144104	0,145098
512000	0,298233	0,347665	0,293722	0,300831	0,299069

Odchylenie standardowe:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000015	0,000008	0,000038	0,000016	0,00001
2000	0,000035	0,000037	0,000035	0,000021	0,000029
4000	0,00006	0,000052	0,000055	0,00004	0,000047
8000	0,000074	0,000064	0,000031	0,000063	0,000039
16000	0,0001	0,000102	0,000251	0,000098	0,000089
32000	0,000136	0,000134	0,00019	0,001424	0,000102
64000	0,000218	0,000183	0,000101	0,000213	0,000348
128000	0,000285	0,000711	0,00089	0,000753	0,000855
256000	0,002314	0,001498	0,00083	0,000661	0,001988
512000	0,0014	0,002413	0,00102	0,003395	0,001276

ALGORYTM HEAP SORT dla danych wzrastających:

- Liniowo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-kształtne
1000	0	0	0,00156768	0	0
2000	0	0	0,00161402	0,0015692	0
3000	0,00156781	0	0,00257403	0	0,00060167
4000	0,00317971	0,00158261	0,00322333	0,00160792	0,00417778
5000	0,00414756	0,00158191	0,0031921	0,00113407	0,00322057
6000	0,00223604	0,00321919	0,00298085	0,00289243	0,00358917
7000	0,00497215	0,00318153	0,00321198	0,00318508	0,00457768
8000	0,00595694	0,00800236	0,0063493	0,00797767	0,00025593
9000	0,00616496	0,00483477	0,00474784	0,0047844	0,00336995
10000	0,00642851	0,00359569	0,00481287	0,00573873	0,00650438

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0	0	0,00470304	0	0
2000	0	0	0,00484206	0,0047076	0
3000	0,00470343	0	0,00458839	0	0,00091907
4000	0,00636028	0,00474783	0,00644666	0,00482376	0,00604762
5000	0,00517664	0,00474573	0,00638469	0,00161096	0,00644116
6000	0,00221302	0,00643838	0,00598254	0,00582472	0,00592924
7000	0,00639163	0,00636326	0,00642399	0,00637029	0,00701273
8000	0,00743681	0,00800323	0,00715077	0,00797797	0,00076779
9000	0,00558289	0,00614253	0,00725282	0,00730897	0,0062634
10000	0,00823869	0,00623718	0,00735189	0,00690678	0,00762781

- Wykładniczo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00160556	0,00139591	0	0	0,00020054
2000	0	0,00157266	0,00096268	0,00020082	0,00059336
4000	0,00393348	0,00157312	0,00372656	0,00175609	0,00282609
8000	0,00473475	0,00518664	0,00320166	0,0031757	0,00708286
16000	0,01108048	0,00811482	0,01153127	0,01604059	0,00958069
32000	0,02340539	0,01892323	0,02041162	0,01895362	0,02250569
64000	0,04671547	0,04281955	0,03924467	0,04162922	0,03985343
128000	0,03957054	0,03184783	0,0337121	0,03188069	0,03489433
256000	0,08414661	0,06802509	0,07141166	0,07317401	0,0767531
512000	0,18330639	0,1497408	0,15229025	0,15434598	0,15762926

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00481668	0,00418773	0	0	0,00060162
2000	0	0,00471798	0,00158363	0,00060246	0,00130668
4000	0,00481657	0,00471936	0,00483305	0,0014066	0,00459281
8000	0,00255577	0,00718931	0,0064035	0,00635179	0,0059927
16000	0,00730797	0,00779005	0,00617108	0,00361547	0,00782346
32000	0,00691585	0,00685556	0,00810003	0,00772014	0,00711857
64000	0,00603076	0,00579647	0,00773813	0,00709924	0,00796684
128000	0,00804481	0,00030975	0,00490954	0,00033534	0,00618924
256000	0,00719256	0,0073771	0,00792804	0,00763602	0,00628311
512000	0,00761168	0,00782118	0,00753567	0,0072783	0,00490713

ALGORYTM QUICK SORT REKURENCYJNY dla danych wzrastających:

- Liniowo:

Średni czas:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000102	0,000211	0,001313	0,001465	0,000101
2000	0,000187	0,000215	0,003986	0,004289	0,000179
3000	0,000301	0,000267	0,008826	0,008797	0,000301
4000	0,000426	0,000364	0,015661	0,01568	0,000404
5000	0,000572	0,000462	0,024532	0,024113	0,000537
6000	0,000703	0,000566	0,035561	0,034641	0,000653
7000	0,000846	0,000682	0,047652	0,046867	0,000788
8000	0,00098	0,000797	0,062795	0,06144	0,000914
9000	0,001077	0,000887	0,079062	0,076848	0,001053
10000	0,001278	0,00101	0,097102	0,096082	0,001182

Odchylenie standardowe:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
1000	0,000004	0,000005	0,000041	0,000292	0,00001
2000	0,000009	0,000007	0,000069	0,000281	0,000013
3000	0,000003	0,000014	0,000125	0,000126	0,00003
4000	0,000021	0,00001	0,000148	0,000372	0,000017
5000	0,000034	0,000007	0,000421	0,000199	0,000049
6000	0,000037	0,00002	0,000819	0,000297	0,000026
7000	0,000046	0,000026	0,000248	0,000279	0,000023
8000	0,000055	0,000031	0,000697	0,000898	0,000037
9000	0,000033	0,000013	0,000806	0,000333	0,00002
10000	0,000069	0,000024	0,000304	0,001081	0,000029

- Wykładniczo:

Średni czas:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
10	0	0,000001	0,000001	0	0
20	0,000001	0,000001	0,000001	0,000001	0,000001
40	0,000002	0,000003	0,000003	0,000004	0,000002
80	0,000004	0,000006	0,000008	0,00001	0,000004
160	0,000009	0,000011	0,000029	0,00003	0,000008
320	0,000021	0,000025	0,000119	0,000109	0,000019
640	0,000055	0,000048	0,000417	0,000444	0,000047
1280	0,000125	0,000103	0,001633	0,00166	0,000107
2560	0,000268	0,000225	0,006443	0,006458	0,000257
5120	0,000623	0,000487	0,025682	0,025329	0,000559
10240	0,001338	0,001034	0,103199	0,099427	0,001255

20480	0,002915	0,002277	0,414512	0,392542	0,002669

Odchylenie standardowe:

n	a-kształtne	losowe	malejące	rosnące	v-kształtne
10	0	0,000001	0,000001	0,000001	0
20	0	0,000001	0,000001	0,000001	0
40	0	0,000001	0,000001	0	0,000001
80	0,000001	0,000001	0,000002	0,000002	0,000001
160	0,000002	0,000002	0,000003	0,000004	0,000001
320	0,000003	0,000008	0,000013	0,000006	0,000001
640	0,000013	0,000006	0,000017	0,000036	0,000007
1280	0,00001	0,000005	0,000036	0,000049	0,000005
2560	0,000004	0,000008	0,000084	0,000083	0,000032
5120	0,000031	0,000012	0,00034	0,000163	0,000015
10240	0,000043	0,000048	0,000948	0,000296	0,000063
20480	0,00014	0,00007	0,003574	0,001851	0,000048

ALGORYTM QUICK SORT ITERACYJNY dla danych wzrastających:

- Liniowo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0	0,006193	0,00377	0,00574657	0,00158521
2000	0,001609	0,026475	0,008922	0,02223593	0,00777938
3000	0,000194	0,053933	0,022911	0,04747036	0,00960268
4000	0,001407	0,099251	0,042226	0,08203185	0,02381656
5000	0,003134	0,152491	0,061604	0,1312004	0,03097218
6000	0,003168	0,213908	0,088363	0,18783594	0,04352888
7000	0,003454	0,292492	0,115333	0,2506502	0,05053252
8000	0,002949	0,370871	0,148346	0,33159368	0,09602693
9000	0,004683	0,457667	0,181889	0,41261493	0,10891557
10000	0,004407	0,566234	0,221319	0,51389976	0,12020702

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0	0,006346	0,001364	0,003904	0,0017
2000	0,004828	0,006759	0,007426	0,007786	0,007841
3000	0,000582	0,007104	0,007396	0,004257	0,007841
4000	0,004221	0,008345	0,007729	0,006773	0,007836
5000	0,00627	0,009287	0,010034	0,005489	0,009531
6000	0,006337	0,006724	0,008443	0,004596	0,010377
7000	0,006277	0,014445	0,006514	0,010638	0,010806
8000	0,00472	0,006262	0,004039	0,004027	0,014362
9000	0,007239	0,00994	0,008388	0,008825	0,023146
10000	0,006822	0,01677	0,007286	0,010882	0,012129

- Wykładniczo:

Średni czas:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00161289	0,00786923	0,00193939	0,00313688	0,00156799
2000	0,00040165	0,02675421	0,0098251	0,02201051	0,00785488
4000	0,00158629	0,09808743	0,04067291	0,08210162	0,02512536
8000	0,00476361	0,36880706	0,14766019	0,32855175	0,0819281
16000	0,00790289	1,34878679	0,53436536	1,30995645	0,32380492
32000	0,01430054	4,71932445	1,69891248	5,29325019	1,33836755
64000	0,0309592	16,12595972	4,65577061	21,26166306	4,59889893

Odchylenie standardowe:

n	losowe	rosnace	malejace	a-kształtne	v-ksztaltne
1000	0,00483867	0,00751327	0,00187879	0,00627376	0,00470397
2000	0,0008033	0,00721322	0,00690616	0,00770765	0,00785489
4000	0,00412877	0,00610266	0,0079695	0,00615842	0,00768996
8000	0,00645446	0,01156294	0,00628017	0,01001427	0,01834479
16000	0,0079031	0,0176973	0,01745322	0,01380854	0,05246604
32000	0,00486128	0,06616304	0,00943897	0,0774196	0,26552748
64000	0,00423941	0,08317215	0,04653711	0,27552925	0,65062504